



Today's agenda

- ↳ No. of Connected Component
- ↳ Topological sort



AlgoPrep



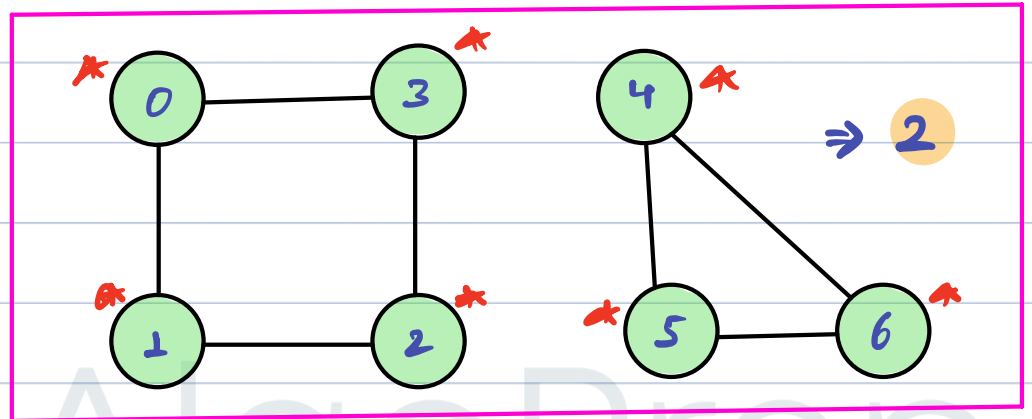
Q) Connected Components

↳ Given undirected graph, find no. of Connected Components.

Note: A component is said to be connected, if from every node we can visit all nodes inside that component.

Exo

0	1	2	3	4	5	6
T	T	T	T	T	T	T



graph

0	1	2	3	4	5	6
1	2	1	2	6	4	4
3	0	3	0	5	6	5



//Pseudo code

↳ multi function dfs

```
int main ( ) {
```

```
    int n =
```

```
    int m =
```

```
    List<List<Integer>> graph = Construction();
```

```
    int ans = 0;
```

```
    boolean[] vis = new boolean[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (vis[i] == false) {
```

```
            vis[i] = true;
```

```
            dfs(graph, vis, i);
```

```
            ans++;
```

```
        }
```

```
    }
```

```
    return ans;
```

```
}
```

```
void dfs (List<List<Integer>> graph, boolean vis[], int src {
```

```
    List<Integer> nbos = graph.get(src);
```

```
    for (int v : nbos) {
```

```
        if (vis[v] == false) {
```

```
            vis[v] = true;
```

```
            dfs(graph, vis, v);
```

```
        }
```

```
    }
```

```
}
```

T.C: $O(V+E)$

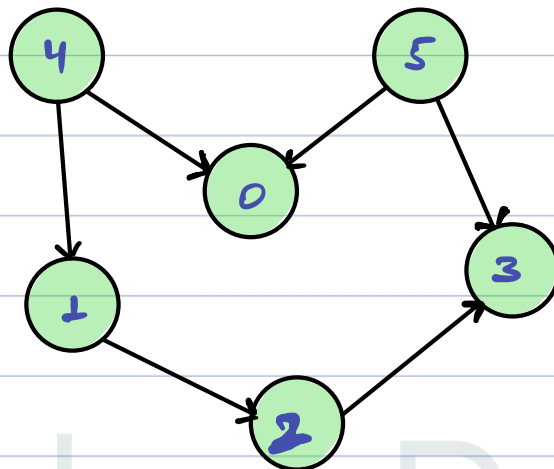
S.C: $O(V)$



Q) Topological Sort

↳ Give the linear ordering of graph such that for every directed edge (u, v) , vertex u comes before v in the ordering. { DAG $\rightarrow$ Directed, Acyclic graph only }

Ex:



TS: 1 3 2 0 4 5 ~~xx~~

TS: 4 0 5 1 3 2 ~~xx~~

There can be multiple topological sort of same graph.

TS: 4 5 0 1 2 3 ✓

TS: 5 4 0 1 2 3 ✓

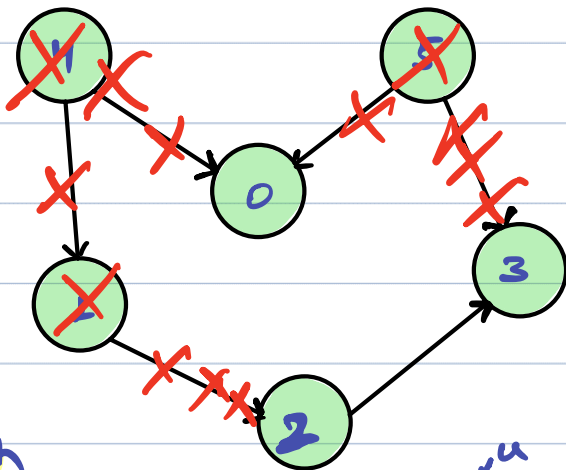


// Kahn's algo

no. of incoming
edge of a
node.

indegree:

0	1	2	3	4	5
2 0	1 0	0 1	0 2	0	0



q: 1 5 4 0 2 3

graph

0	1	2	3	4	5
	2	3		1	3
				0	0

Topological sort!

4 5 1 0 2 3

// Pseudo code

```

void topologicalSort (List<List<Integer>> graph,
                     int n, int m) {

```

```

    int[] indegree = new int[n];

```

```

    for (int u = 0; u < n; u++) {

```

```

        List<Integer> nbos = graph.get(u);

```

```

        for (int v : nbos) {

```

```

            indegree[v]++;

```

```

        }

```

```

    }

```



Queue < Integer > q;

T.C: $O(V+E)$

S.C: $O(V)$

```
for (int i = 0; i < n; i++) {  
    if (indegree[i] == 0) { q.add(i);  
}
```

```
while (q.size() > 0) {  
    int rem = q.remove();  
    s.o.p(rem);
```

```
List < Integer > nbrs = graph.get(rem);
```

```
for (int v : nbrs) {  
    indegree[v] --;  
    if (indegree[v] == 0) {  
        q.add(v);  
    }  
}
```

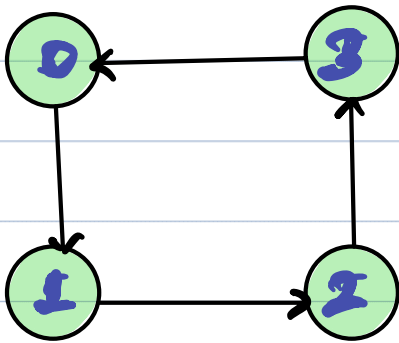
```
}
```

```
}
```

```
}
```



① what happens if graph is cyclic?

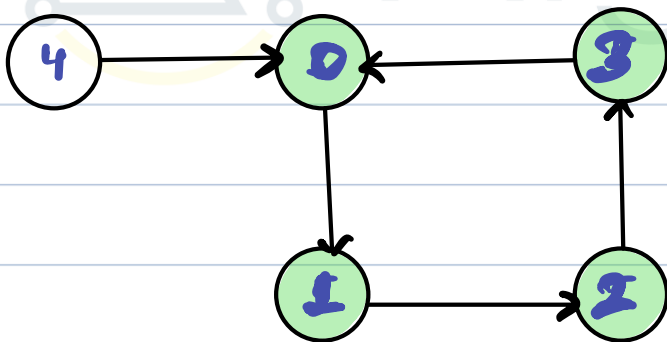


indegree:

0	1	2	3
1	1	1	1

$u \rightarrow v$

$u < v$



indegree:

0	1	2	3	4
2	1	1	1	0

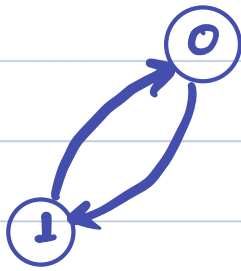
2: 1
sum=4

4

→ in cyclic graph, dependency is also cyclic. nowhere to start from.



ii) why not undirected graph?



→ Same issue as cyclic graph.

Q) Detect cycle in directed graph.

directed graph

acyclic

topological sort
is valid.

cyclic

↳ you can't complete
the topological ordering.

// Pseudo code

```
void topologicalSort (List<List<Integer>> graph,  
int n, int m) {
```

```
int[] indegree = new int[n];
```

```
for (int u = 0; u < n; u++) {
```

```
    List<Integer> nbors = graph.get(u);
```

```
    for (int v : nbors) {
```

```
        indegree[v]++;
```




```
Queue < Integer > q;
```

T.C: $O(V+E)$

S.C: $O(V)$

```
for (int i=0; i<n; i++) {  
    if (indegree[i] == 0) { q.add(i);  
}
```

```
int count = 0;
```

```
while (q.size() > 0) {  
    int rem = q.remove();  
    s.op(rem);  
    count++;
```

```
List < Integer > nbrs = graph.get(rem);
```

```
for (int v: nbrs) {  
    indegree[v]--;  
    if (indegree[v] == 0) {  
        q.add(v);  
    }  
}
```

```
}
```

```
if (count == n) { s.op("acyclic"); }  
else { s.op("cyclic"); }
```



AlgoPrep